Brief — Report vendite e clienti (Fake Store)

Contesto

Il team commerciale ha bisogno di due report analitici sull'e-commerce, basati sui dati pubblici di Fake Store API. Tu sei responsabile di realizzare un'applicazione che li produca.

Deliverable

1. **Repository Git nuovo** (remoto pubblico o privato condiviso con i docenti), dedicato esclusivamente a questa esercitazione. Non riusare un progetto già esistente del corso.
2. Codice funzionante che, all'avvio, recupera i dati dall'API e stampa in console (o scrive su file) i due report descritti sotto, in JSON. La forma dell'oggetto (annidamento, nomi delle chiavi, organizzazione delle sezioni) la progetti tu: l'unico vincolo è che ogni report esponga **tutti gli attributi** elencati per quel report.
3. `README.md` con: come installare le dipendenze, come avviare il programma, e una frase su come hai definito il periodo «ultimo mese» rispetto ai dati.

Stack, struttura delle cartelle, librerie e stile di codice sono a tua scelta, purché il risultato sia eseguibile localmente con istruzioni chiare.

Fonte dati

Base URL: `https://fakestoreapi.com/`

Risorse rilevanti (documentazione: https://fakestoreapi.com/docs):

Endpoint	Contenuto utile
`GET /products`	Catalogo: id, titolo, prezzo, …
`GET /carts`	Ordini/carrelli: `userId`, `date`, elenco `{ productId, quantity }`
`GET /users`	Anagrafica clienti

Report 1 — Top 3 prodotti più venduti

Criterio di ranking: unità vendute, non il fatturato.

Output atteso. Il report deve essere un JSON che, per entrambi i periodi (all time e ultimo mese), elenca fino a tre prodotti in classifica. Per ogni prodotto in classifica devono essere presenti, in qualunque annidamento tu scelga:

- la posizione in classifica;
- l'identificativo del prodotto;
- il titolo;
- la categoria;
- il prezzo unitario di catalogo;
- le unità vendute nel periodo;

- il fatturato del periodo (prezzo unitario × unità vendute).

Inoltre il report deve dichiarare:

- un identificativo o etichetta che lo distingua come report prodotti;
- il momento in cui è stato generato;
- la data dell'ordine più recente usata come ancoraggio per «ultimo mese»;
- l'intervallo temporale considerato per l'ultimo mese (inizio e fine); per l'all time puoi omettere l'intervallo o esplicitarlo come assente — scegli tu, ma la scelta deve essere coerente e leggibile.

Report 2 — Top 3 clienti per volume di acquisto

Criterio di ranking: spesa totale, non il numero di ordini.

Output atteso. Il report deve essere un JSON che, per entrambi i periodi (all time e ultimo mese), elenca fino a tre clienti in classifica. Per ogni cliente in classifica devono essere presenti:

- la posizione in classifica;
- l'identificativo utente;
- lo username;
- il nome completo (nome e cognome come restituiti dall'API, uniti da uno spazio, senza capitalizzazione extra);
- l'email;
- il numero di ordini (carrelli) nel periodo;
- il totale pezzi acquistati nel periodo (somma delle quantità);
- la spesa totale nel periodo (somma di prezzo × quantità).

Come nel Report 1, il JSON deve anche dichiarare etichetta/identificativo del report, momento di generazione, data di ancoraggio per «ultimo mese» e intervallo dell'ultimo mese. Stessa libertà sulla forma: conta che tutti questi attributi siano recuperabili dal JSON, non come li organizzi.

Criteri di accettazione

Il lavoro è considerato completo se:

- il repo è nuovo e contiene istruzioni di avvio funzionanti;
- i due report sono JSON e contengono tutti gli attributi richiesti sopra (la struttura la definisci tu);
- ranking, periodi e tie-break rispettano le regole di questo brief;
- un prodotto o un utente assente dal catalogo/anagrafica ma presente in un carrello non fa crashare il programma (decidi tu una strategia sensata e documentala in una riga nel README).

Fuori scope

UI grafica, autenticazione, scrittura su database, deploy, test automatici obbligatori. Puoi aggiungerli, ma non sono richiesti per la consegna.